



**ABV-Indian Institute of Information Technology and
Management, Gwalior**

Course: Machine Learning Techniques (MLT)

Movie Recommendation System using Collaborative Filtering and Differential Privacy

Team Members:

Ushosree Raha (2025MS027)

Yazhini K (2025MS032)

Instructor: Dr. Santosh Singh Rathore

Date of Submission: November 18, 2025

Certificate

This is to certify that the project entitled “**Movie Recommendation System using Collaborative Filtering and Differential Privacy**” has been conducted by Ushosree Raha (2025MS027) and Yazhini K (2025MS032) under the guidance of Dr. Santosh Singh in partial fulfillment of requirements for the course Machine Learning Techniques (MLT).

Instructor Signature

Acknowledgements

We gratefully acknowledge the support of Dr. Santosh Singh for his guidance through this project. We also thank GroupLens for providing the MovieLens dataset and the open-source community for the tools used in this work.

Abstract

Recommender systems are essential for personalized experiences across media and commerce platforms. Collaborative filtering (CF) is a widely used approach that exploits user-item rating patterns to predict unknown preferences. However, training recommender systems requires user data that can be privacy-sensitive. This project implements and compares naive CF and Neural Collaborative Filtering (NCF) on the MovieLens-1M dataset and studies Differential Privacy (DP) defenses: Gaussian noise injection and DP-SGD. We perform exploratory data analysis (EDA), implement models, and evaluate the privacy-utility trade-off. Results show NCF outperforms naive CF in prediction accuracy; DP increases training loss but can still yield useful recommendations at moderate privacy levels. We discuss implementation details, experimental results, and future directions such as federated learning and adaptive privacy mechanisms.

Contents

1	Introduction	1
2	Dataset Description and Exploratory Data Analysis	2
2.1	Dataset Overview	2
2.2	Preprocessing Summary	2
2.3	Exploratory Visualizations and Interpretation	3
3	Methodology	5
3.1	Feature Engineering and Normalization	5
3.2	Train/Test Splitting Strategy	5
3.3	Algorithmic Implementation	6
3.3.1	Naive Collaborative Filtering	6
3.3.2	Neural Collaborative Filtering	6
3.3.3	Differential Privacy	6
3.4	Evaluation	6
4	Model Training and Implementation	7
4.1	Naive Collaborative Filtering	7
4.2	Neural Collaborative Filtering (NCF)	7
4.3	Differential Privacy Implementations	8
4.4	Training Procedure and Hyperparameters	9
5	Results	10
5.1	Training and Privacy Comparisons	10
6	Conclusion and Future Work	12
6.1	Conclusion	12
6.2	Future Work	12
6.3	Contributions	12

Chapter 1

Introduction

Recommender systems are ubiquitous: they help users discover movies, products, news, and music suited to their tastes by analyzing historical behavior. Collaborative Filtering (CF) is a prominent technique that predicts a user’s interests by leveraging the ratings of similar users or items, rather than relying on content descriptions. This project centers on building a CF-based movie recommender using the Movie dataset and further investigates methods for protecting user privacy when learning from such data.

The motivation for deploying privacy-preserving recommendation models is twofold. First, user activity data such as ratings, viewing histories, or check-ins can reveal sensitive information about individuals. Second, regulatory and ethical considerations increasingly require systems that minimize privacy leakage. Differential Privacy (DP) offers a mathematical framework to bound the influence of any single user’s data on the trained model outputs.

This work focuses on three objectives. The first objective is to perform a thorough exploratory data analysis (EDA) on the MovieLens-1M dataset; this identifies important distributional properties like sparsity and genre imbalance. The second objective is to implement baseline CF methods and a Neural Collaborative Filtering (NCF) model, and to evaluate their prediction quality using standard metrics. The third objective is to apply DP mechanisms (noise addition and DP-SGD) and to quantify how privacy changes the utility of the trained models. Each objective contributes to a clear pipeline for privacy-aware recommendation.

The scope includes rating prediction and empirical evaluation on standard metrics like RMSE and MAE. We do not attempt online deployment or user studies; instead, we focus on offline experiments that can be reproduced by other researchers. The remainder of the report follows the lab instructions by covering dataset & EDA, methodology & preprocessing, models, experimental results, discussion, and conclusions.

Chapter 2

Dataset Description and Exploratory Data Analysis

2.1 Dataset Overview

The MovieLens-1M dataset contains approximately 1,000,209 ratings from 6,040 users on 3,900 movies. Each entry contains a user ID, movie ID, a rating from 1 to 5, and a timestamp. The dataset also contains movie metadata, notably titles and genres, which are useful for understanding the content distribution. Because the data is real user feedback, it exhibits typical properties of human-generated ratings: skew towards the middle values, seasonal activity, and strong popularity bias for certain items.

2.2 Preprocessing Summary

Data preprocessing involved parsing timestamps into year/month/day, label-encoding user and movie IDs for embedding layers, and normalizing ratings for use with MSE loss in the neural model. We split the data into 80% training and 20% test sets keeping timestamp ordering where appropriate to avoid leakage. No aggressive outlier removal was necessary since ratings are bounded.

2.3 Exploratory Visualizations and Interpretation

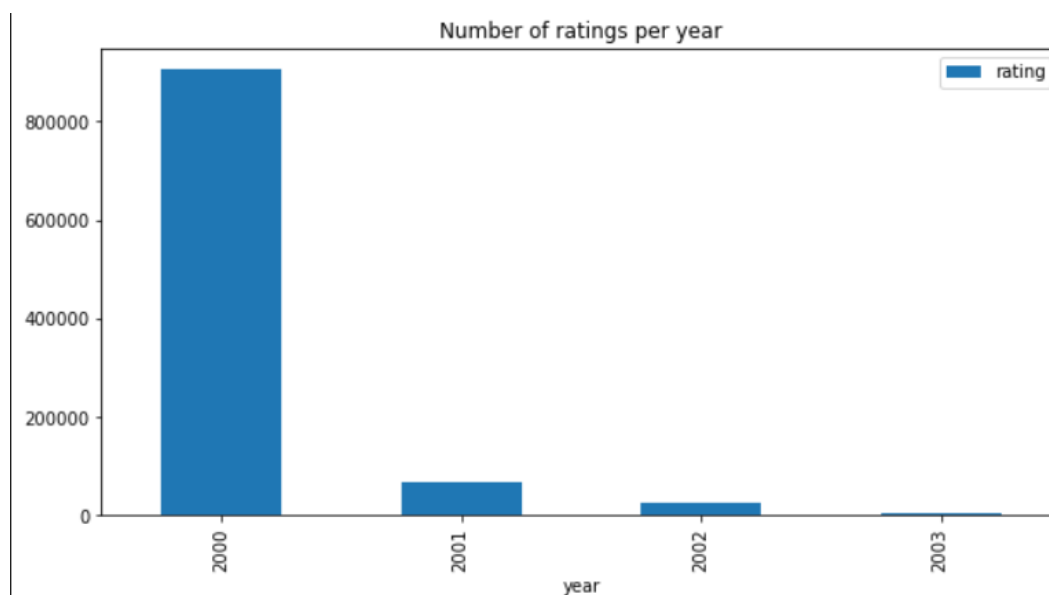


Figure 2.1: Number of ratings per year (2000–2003).

This bar chart displays the distribution of movie ratings across the years 2000 to 2003. A clear observation is that the overwhelming majority of ratings were submitted in the year 2000, exceeding 850,000 entries, while subsequent years show significantly fewer contributions. This imbalance results from the dataset collection timeline: MovieLens received many early ratings after launch and then stabilized. The temporal concentration means that models trained on the data reflect early user behavior more strongly than later changes. Practically, this suggests either using time-aware models or acknowledging temporal bias when interpreting results. It also implies that validation should carefully account for the time dimension to avoid overly optimistic evaluations.

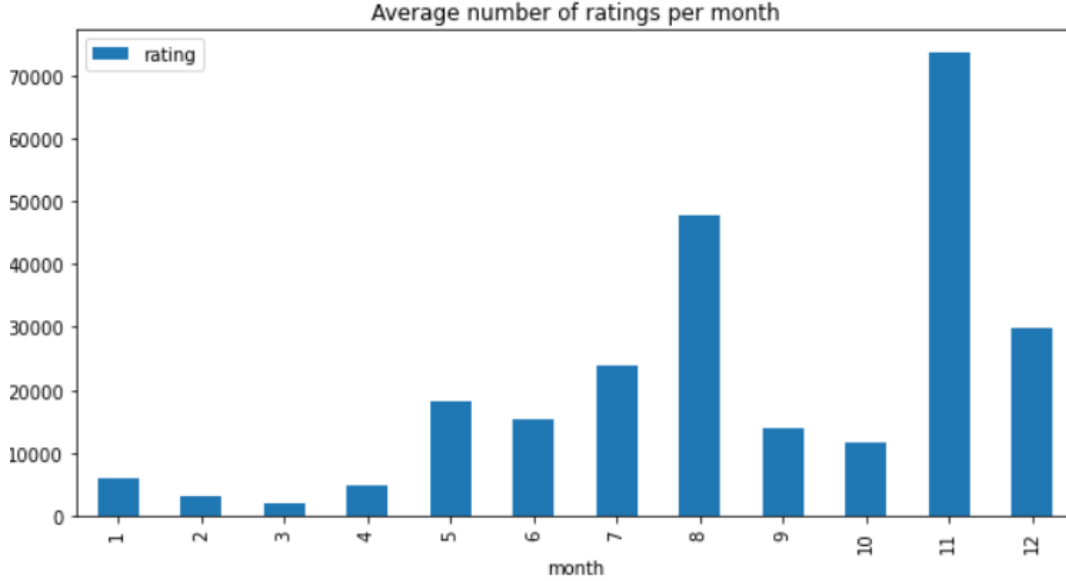


Figure 2.2: Average number of ratings per month.

This bar graph shows the average number of ratings per month across the dataset period. Activity peaks in months like August and November, likely corresponding to holiday or leisure periods. Conversely, some months show markedly fewer ratings. Seasonal trends matter because recommendation models trained on high-activity periods might generalize poorly to low-activity months. Recognizing these patterns helps in planning time-split evaluations and in choosing models that can adapt to varying user activity levels.

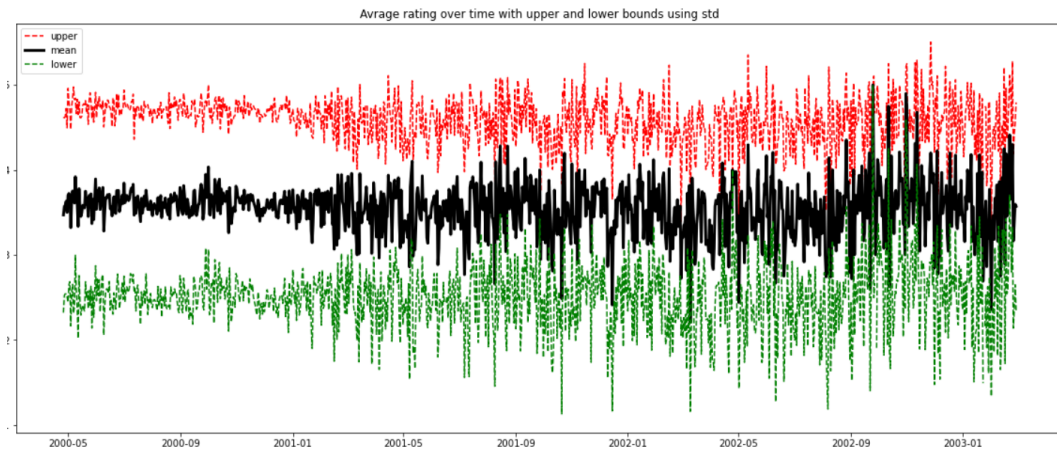


Figure 2.3: Average rating over time with upper and lower standard deviation bounds.

The mean rating stays mostly stable over time, moving only slightly. The upper and lower bands show how much user opinions vary at each point. Wider bands mean users disagreed more, while narrow bands mean they rated similarly.

Chapter 3

Methodology

Our pipeline follows a stepwise approach: data acquisition and EDA , preprocessing and feature encoding , baseline algorithm implementation, neural model design and training, and finally DP integration and evaluation. This modular design ensures reproducibility and isolates the effects of privacy mechanisms from model architecture choices.

3.1 Feature Engineering and Normalization

Key preprocessing steps included label-encoding user and item identifiers for embedding layers and scaling ratings where appropriate. For neural models we normalized ratings to a 0–1 range for stable training with MSE; for evaluation we converted predictions back to the original 1–5 scale. Additional derived features (such as year or genre flags) were reserved for future work but were not necessary for the baseline experiments.

3.2 Train/Test Splitting Strategy

We used an 80/20 random split for training and testing for fairness in baseline comparisons. In some runs we also experimented with time-based splits to reflect realistic deployment; those splits help evaluate how well the model generalizes to future data. Care was taken to avoid leakage between splits.

3.3 Algorithmic Implementation

3.3.1 Naive Collaborative Filtering

Collaborative filtering using Pearson correlation on the MovieLens dataset works by identifying users who rate movies in similar ways. First, we build a user-item rating matrix from the dataset. Then, for a target user, we calculate the Pearson correlation between their ratings and those of all other users to measure similarity based on rating patterns rather than raw values. Users with high positive correlation are considered closer neighbors. Finally, we generate movie recommendations by taking weighted averages of these neighbors' ratings, giving more weight to users with higher correlation. This approach helps predict how much a target user may like a movie they haven't seen by leveraging the behavior of similar users.

3.3.2 Neural Collaborative Filtering

NCF uses neural networks to map user IDs and item IDs into dense embedding vectors, which capture latent features. These embeddings are then combined— by concatenation and passed through multiple fully connected layers that learn nonlinear relationships between users and items. The architecture shown 4.1 uses embeddings, reshaping, concatenation, dense layers, activations, and dropout to gradually transform the user-item pair into a predicted rating. Because the model learns its own interaction function rather than using a fixed formula, NCF can capture richer patterns and often achieves better recommendation accuracy than traditional collaborative filtering methods.

3.3.3 Differential Privacy

ϵ -differential privacy and (δ, ϵ) -differential privacy were implemented. The modified data was then fed to NCF model to obtain movie recommendations.

3.4 Evaluation

The results of Collaborative Filtering and Neural Collaborative Filtering with and without the addition of differential privacy were evaluated.

Chapter 4

Model Training and Implementation

4.1 Naive Collaborative Filtering

Naive CF uses similarity measures (cosine similarity) between items or users. For item-based CF, we compute pairwise similarities between item columns in the user-item matrix and predict a user's unknown rating using a weighted average of the user's ratings on similar items. This method is interpretable and fast for moderate datasets but degrades when co-rating frequency is low.

4.2 Neural Collaborative Filtering (NCF)

The NCF model uses embedding layers to map user and item IDs to dense vectors and combines them through concatenation followed by dense layers. The architecture used in experiments is shown below.

Figure 4.1 illustrates the complete architecture of the Neural Collaborative Filtering model used for movie rating prediction. It begins with two separate `InputLayer` blocks, accepting user IDs and movie IDs as model inputs. Each ID is passed into an `Embedding` layer, converting sparse integer identifiers into dense 50-dimensional latent vectors. The embeddings are reshaped to match the neural network's expected format before being combined using a `Concatenate` layer. This merged feature vector represents the joint interaction between a specific user and movie. A series of `Dense` layers progressively transform this interaction space to learn increasingly complex patterns. `Dropout` layers are incorporated to reduce overfitting by randomly deactivating neurons during training. Activation functions introduce non-linear transformations essential for capturing real-world rating behaviors. Finally, a `Lambda` layer scales the output back to the valid rating range, enabling the model

The following is the architecture of the neural network that has been used to train the model:

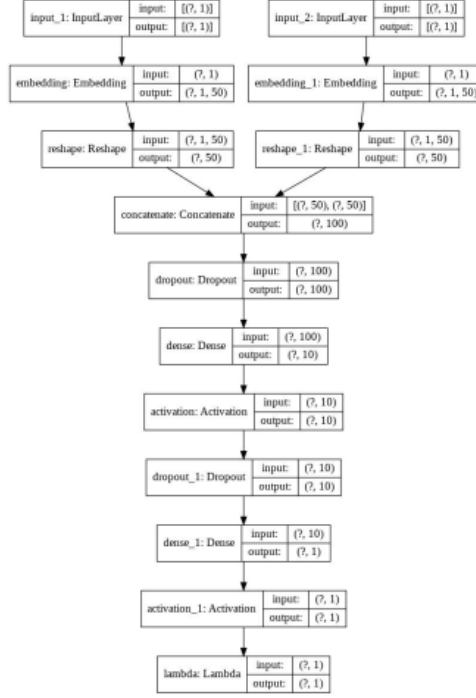


Figure 4.1: Neural Collaborative Filtering architecture (embeddings, concat, dense layers).

to produce accurate predictions. Overall, this architecture provides a powerful and flexible framework for modeling user preferences.

4.3 Differential Privacy Implementations

Data are anonymized using random additions of noise in the selected datasets. In **Differential privacy (DP)**, a randomized function ($\mathcal{F}$) ensure ϵ -DP if $\forall$ real datasets D_1 and D_2 differ at most by 1 tuple and $\forall S \subseteq \text{Range}(\mathcal{F})$,

$$Pr[\mathcal{F}(D_1) \in S] \leq e^\epsilon Pr[\mathcal{F}(D_\epsilon) \in S] \quad (4.1)$$

The above equation ensures strict privacy, but often at the cost of utility. The information leakage in real life is also considered in $(\delta, \epsilon) - DP$ as:

$$Pr[\mathcal{F}(D_1) \in S] \leq e^\epsilon Pr[\mathcal{F}(D_\epsilon) \in S] + \delta \quad (4.2)$$

where δ is the probability of information being accidentally leaked.

Considering these, two DP strategies were implemented:

1. **Gaussian noise addition to ratings:** Add Gaussian noise at various scales (0.5, 1.0, 1.5, 2.0) to the rating values before training. This is a simple data-level approach.
2. **DP-SGD:** Use TensorFlow Privacy to clip per-example gradients and add calibrated Gaussian noise during optimizer updates. This approach gives a formal (ϵ, δ) guarantee under assumptions.

4.4 Training Procedure and Hyperparameters

The training procedure is designed to be simple, robust, and easy to reproduce. We begin by initializing the model weights using standard Xavier/Glorot initialization for dense layers and small random initialization for embeddings. Training proceeds using the Adam optimizer because it adapts the learning rate per parameter and typically converges faster than plain SGD for our architectures. For each training run we use minibatch training: data is shuffled at the start of each epoch and divided into batches (commonly 128 or 256 examples per batch). For neural models the loss function chosen is Mean Squared Error (MSE) as the task is rating regression; for some baseline comparisons we also report Mean Absolute Error (MAE) because it is more interpretable in terms of average rating deviation.

Hyperparameters were selected after a few controlled trials on a validation split. Important hyperparameters and our typical choices are:

- **Embedding dimension:** 50. This value balances expressiveness and computational cost; smaller dims underfit while much larger dims add parameters without clear gains.
- **Batch size:** 256. This provides stable gradient estimates and efficient GPU/CPU usage.
- **Learning rate:** 0.001 (Adam default). We also test 0.0005 and 0.0001 for sensitive DP runs because noisy gradients require smaller steps.
- **Dropout:** 0.2–0.4 on dense layers to reduce overfitting.
- **Number of epochs:** up to 20 with early stopping on validation loss (patience 3). This prevents long overfitting runs while letting the model converge.
- **Noise multiplier (DP-SGD):** varied (e.g., 0.5–2.0) to achieve different privacy budgets; we measure the corresponding (ϵ, δ) using the moments accountant.

Chapter 5

Results

5.1 Training and Privacy Comparisons

Results:

The following are the results obtained

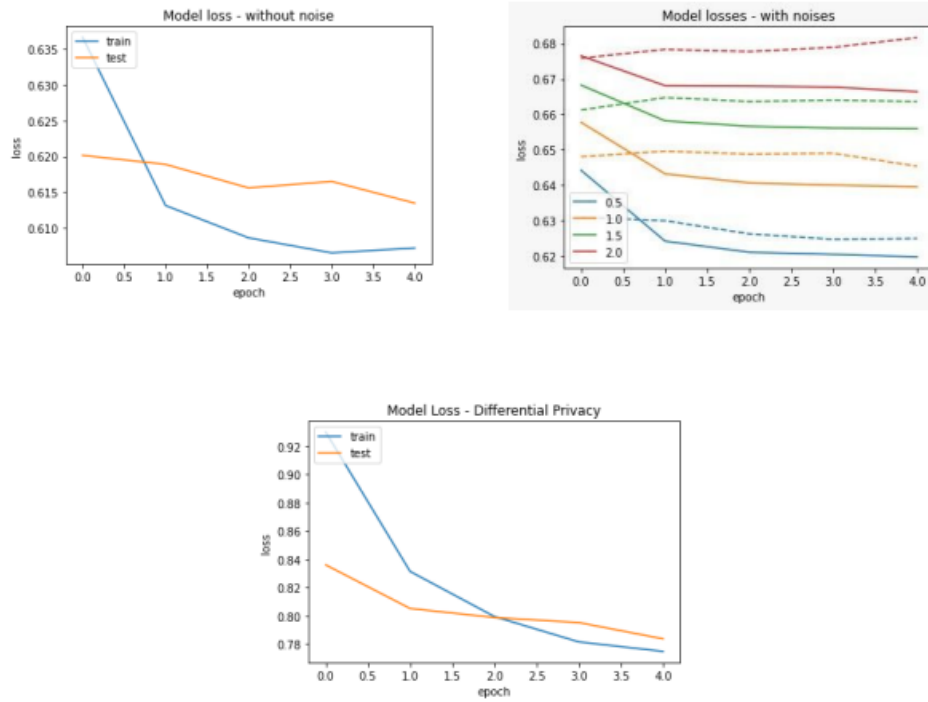


Figure: Comparison of results between different approaches

Figure 5.1: Loss curves for models without noise, with Gaussian noise, and with DP-SGD.

This combined figure compares the training and testing loss behavior across three scenarios: standard training without noise, training with varying levels of Gaussian noise, and training with Differential Privacy (DP-SGD). In the no-noise graph, both train and test losses reduce smoothly, demonstrating stable learning and good generalization. The graph with noise levels shows how increasing noise (0.5, 1.0, 1.5, 2.0) progressively degrades model performance, flattening or even increasing loss curves due to disruption of training signals. Models trained with large noise values struggle to converge as the injected randomness overwhelms meaningful gradients. The Differential Privacy graph shows higher initial loss compared to the no-noise case but eventually reaches an acceptable range, indicating controlled learning under strict privacy constraints. Together, these curves illustrate the trade-off between accuracy and privacy and show that moderate noise preserves utility while excessive noise weakens modeling capacity.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project demonstrated a complete pipeline for building recommender systems with attention to privacy. Naive CF provides a simple baseline, while Neural Collaborative Filtering captures richer user–item interactions via embeddings and dense layers. Differential Privacy, applied either by noise injection or via DP-SGD, increases model loss but can be tuned to maintain acceptable utility. DP-SGD is generally more robust than naive data perturbation.

6.2 Future Work

The use of LDP shall be explored further. Future directions include combining federated learning with DP. In addition, expanding experiments across multiple datasets and conducting user studies would strengthen conclusions about real-world utility. In addition to semantic privacy, syntactic privacy techniques, even hybrid methods shall be explored further.

6.3 Contributions

Name	Roll No.	Contribution
Ushosree Raha	2025MS027	EDA, baseline CF, report writing
Yazhini K	2025MS032	NCF implementation, DP experiments, figures

Bibliography

- [1] D. Yang, B. Qu and P. Cudré-Mauroux, “Privacy-Preserving Social Media Data Publishing for Personalized Ranking-Based Recommendation,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 3, pp. 507–520, 2019.
- [2] X. Li and D. Li, “An Improved Collaborative Filtering Recommendation Algorithm and Recommendation Strategy,” *Mobile Information Systems*, vol. 2019, Article ID 3560968, 11 pages, 2019.
- [3] L. Wang, D. Yang, X. Han, T. Wang, D. Zhang and X. Ma, “Location Privacy-Preserving Task Allocation for Mobile Crowdsensing with Differential Geo-Obfuscation,” in *Proceedings of the 26th International Conference on World Wide Web*, International World Wide Web Conferences Steering Committee, Geneva, Switzerland, pp. 627–636, 2017. doi: 10.1145/3038912.3052696.
- [4] F. M. Harper and J. A. Konstan, “The MovieLens Datasets: History and Context,” *ACM Transactions on Interactive Intelligent Systems*, 2015.
- [5] G. M. Weiss, “Recommender Systems: The Basics,” *Surprise library documentation*, 2014.
- [6] C. Dwork, F. McSherry, K. Nissim and A. Smith, “Calibrating noise to sensitivity in private data analysis,” in *TCC*, 2006.
- [7] A. Majeed and S. O. Hwang, ”A Data-Centric ℓ -Diversity Model for Securely Publishing Personal Data With Enhanced Utility,” in *IEEE Transactions on Big Data*, vol. 11, no. 5, pp. 2278-2295, Oct. 2025, doi: 10.1109/TBDATA.2024.3524832.
- [8] A. Tobar Nicolau, J. Parra-Arnau, J. Forné and V. Torra, ”Uncoordinated Syntactic Privacy: A New Composable Metric for Multiple, Independent Data Publishing,” in *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 3362-3373, 2025, doi: 10.1109/TIFS.2025.3551645.